

Configuration > next.config.js > rewrites

rewrites

Last updated February 27, 2026

Rewrites allow you to map an incoming request path to a different destination path.

Rewrites act as a URL proxy and mask the destination path, making it appear the user hasn't changed their location on the site. In contrast, [redirects](#) will reroute to a new page and show the URL changes.

To use rewrites you can use the `rewrites` key in `next.config.js`:

```
JS next.config.js

module.exports = {
  async rewrites() {
    return [
      {
        source: '/about',
        destination: '/',
      },
    ],
  },
}
```

Rewrites are applied to client-side routing. In the example above, navigating to `<Link href="/about">` will serve content from `/` while keeping the URL as `/about`.

`rewrites` is an async function that expects to return either an array or an object of arrays (see below) holding objects with `source` and `destination` properties:

`source`: `String` - is the incoming request path pattern.

`destination`: `String` is the path you want to route to.

`basePath`: `false` or `undefined` - if false the basePath won't be included when matching, can be used for external rewrites only.

`locale`: `false` or `undefined` - whether the locale should not be included when matching.

`has` is an array of [has objects](#) with the `type`, `key` and `value` properties.

`missing` is an array of [missing objects](#) with the `type`, `key` and `value` properties.

When the `rewrites` function returns an array, rewrites are applied after checking the filesystem (pages and `/public` files) and before dynamic routes.

When the `rewrites` function returns an object of arrays with a specific shape, this behavior can be changed and more finely controlled, as of `v10.1` of Next.js:

 next.config.js

```
module.exports = {
  async rewrites() {
    return {
      beforeFiles: [
        // These rewrites are checked after headers/redirects
        // and before all files including _next/public files which
        // allows overriding page files
        {
          source: '/some-page',
          destination: '/somewhere-else',
          has: [{ type: 'query', key: 'overrideMe' }],
        },
      ],
      afterFiles: [
        // These rewrites are checked after pages/public files
        // are checked but before dynamic routes
        {
          source: '/non-existent',
          destination: '/somewhere-else',
        },
      ],
      fallback: [
        // These rewrites are checked after both pages/public files
        // and dynamic routes are checked
        {
          source: '/:path*',
          destination: `https://my-old-site.com/:path*`,
        },
      ],
    }
  },
}
```

Good to know: rewrites in `beforeFiles` do not check the filesystem/dynamic routes immediately after matching a source, they continue until all `beforeFiles` have been checked.

The order Next.js routes are checked is:

`headers` are checked/applied

`redirects` are checked/applied

`proxy`

`beforeFiles` rewrites are checked/applied

static files from the [public directory](#), `_next/static` files, and non-dynamic pages are checked/served

`afterFiles` rewrites are checked/applied, if one of these rewrites is matched we check dynamic routes/static files after each match

`fallback` rewrites are checked/applied, these are applied before rendering the 404 page and after dynamic routes/all static assets have been checked. If you use `fallback: true/'blocking'` in `getStaticPaths`, the `fallback` `rewrites` defined in your `next.config.js` will *not* be run.

Rewrite parameters

When using parameters in a rewrite the parameters will be passed in the query by default when none of the parameters are used in the `destination`.

JS next.config.js

```
module.exports = {
  async rewrites() {
    return [
      {
        source: '/old-about/:path*',
        destination: '/about', // The :path parameter isn't used here
      },
    ],
  },
}
```

If a parameter is used in the destination none of the parameters will be automatically passed in the query.

JS next.config.js

```
module.exports = {
  async rewrites() {
    return [
      {
        source: '/docs/:path*',
        destination: '/:path*', // The :path parameter is used here s
```

```
    },  
  ],  
},  
}
```

You can still pass the parameters manually in the query if one is already used in the destination by specifying the query in the `destination`.

JS next.config.js

```
module.exports = {  
  async rewrites() {  
    return [  
      {  
        source: '/:first/:second',  
        destination: '/:first?second=:second',  
        // Since the :first parameter is used in the destination the  
        // will not automatically be added in the query although we c  
        // as shown above  
      },  
    ],  
  },  
}
```

Good to know: Static pages from [Automatic Static](#)

[Optimization](#) or [prerendering](#) params from rewrites will be parsed on the client after hydration and provided in the query.

Path Matching

Path matches are allowed, for example `/blog/:slug` will match

`/blog/first-post` (no nested paths):

JS next.config.js

```
module.exports = {  
  async rewrites() {  
    return [  
      {  
        source: 'blog/:slug',  
        destination: '/blog/:slug',  
        permanent: true,  
      },  
    ],  
  },  
}
```

```

    source: '/blog/:slug',
    destination: '/news/:slug', // Matched parameters can be used
  },
]
},
}

```

The pattern `/blog/:slug` matches `/blog/first-post` and `/blog/post-1` but not `/blog/a/b` (no nested paths). Patterns are anchored to the start: `/blog/:slug` will not match `/archive/blog/first-post`.

You can use modifiers on parameters: `*` (zero or more), `+` (one or more), `?` (zero or one). For example, `/blog/:slug*` matches `/blog`, `/blog/a`, and `/blog/a/b/c`.

Read more details on [path-to-regexp](#) documentation.

Wildcard Path Matching

To match a wildcard path you can use `*` after a parameter, for example `/blog/:slug*` will match `/blog/a/b/c/d/hello-world`:

JS next.config.js

```

module.exports = {
  async rewrites() {
    return [
      {
        source: '/blog/:slug*',
        destination: '/news/:slug*', // Matched parameters can be used
      },
    ]
  },
}

```

Regex Path Matching

To match a regex path you can wrap the regex in parenthesis after a parameter, for example `/blog/:slug(\\d{1,})` will match `/blog/123` but not `/blog/abc`:

 next.config.js

```
module.exports = {
  async rewrites() {
    return [
      {
        source: '/old-blog/:post(\\d{1,})',
        destination: '/blog/:post', // Matched parameters can be used
      },
    ],
  },
}
```

The following characters `(, ), {, }, [, ], |, \, ^, ., :, *, +, -, ?`, `$` are used for regex path matching, so when used in the `source` as non-special values they must be escaped by adding `\\` before them:

 next.config.js

```
module.exports = {
  async rewrites() {
    return [
      {
        // this will match `/english(default)/something` being request
        source: '/english\\(default\\)/:slug',
        destination: '/en-us/:slug',
      },
    ],
  },
}
```

Header, Cookie, and Query Matching

To only match a rewrite when header, cookie, or query values also match the `has` field or don't match the `missing` field can be used. Both the `source` and all `has` items must match and all `missing` items must not match for the rewrite to be applied.

`has` and `missing` items can have the following fields:

`type`: `String` - must be either `header`, `cookie`, `host`, or `query`.

`key`: `String` - the key from the selected type to match against.

`value`: `String` or `undefined` - the value to check for, if undefined any value will match. A regex like string can be used to capture a specific part of the value, e.g. if the value `first-(?<paramName>.*)` is used for `first-second` then `second` will be usable in the destination with `:paramName`.

JS next.config.js

```
module.exports = {
  async rewrites() {
    return [
      // if the header `x-rewrite-me` is present,
      // this rewrite will be applied
      {
        source: '/*:path*',
        has: [
          {
            type: 'header',
            key: 'x-rewrite-me',
          },
        ],
        destination: '/another-page',
      },
      // if the header `x-rewrite-me` is not present,
      // this rewrite will be applied
      {
        source: '/*:path*',
        missing: [
          {
            type: 'header',
            key: 'x-rewrite-me',
          },
        ],
        destination: '/another-page',
      },
      // if the source, query, and cookie are matched,
      // this rewrite will be applied
      {
        source: '/specific/:path*',
        has: [
          {
            type: 'query',
            key: 'page',
            // the page value will not be available in the
            // destination since value is provided and doesn't
            // use a named capture group e.g. (?<page>home)
            value: 'home',
          },
        ],
      },
    ],
  },
}
```



```
    },
    {
      type: 'cookie',
      key: 'authorized',
      value: 'true',
    },
  ],
  destination: '/*:path*/home',
},
// if the header `x-authorized` is present and
// contains a matching value, this rewrite will be applied
{
  source: '/*:path*',
  has: [
    {
      type: 'header',
      key: 'x-authorized',
      value: '(<authorized>yes|true)',
    },
  ],
  destination: '/home?authorized=:authorized',
},
// if the host is `example.com`,
// this rewrite will be applied
{
  source: '/*:path*',
  has: [
    {
      type: 'host',
      value: 'example.com',
    },
  ],
  destination: '/another-page',
},
]
},
}
```

Rewriting to an external URL

► Details

Rewrites allow you to rewrite to an external URL. This is especially useful for incrementally adopting Next.js. The following is an example rewrite for redirecting the `/blog` route of your main app to an external site.

 next.config.js

```
module.exports = {
  async rewrites() {
    return [
      {
        source: '/blog',
        destination: 'https://example.com/blog',
      },
      {
        source: '/blog/:slug',
        destination: 'https://example.com/blog/:slug', // Matched par
      },
    ]
  },
}
```

If you're using `trailingSlash: true`, you also need to insert a trailing slash in the `source` parameter. If the destination server is also expecting a trailing slash it should be included in the `destination` parameter as well.

 next.config.js

```
module.exports = {
  trailingSlash: true,
  async rewrites() {
    return [
      {
        source: '/blog/',
        destination: 'https://example.com/blog/',
      },
      {
        source: '/blog/:path*/',
        destination: 'https://example.com/blog/:path*/',
      },
    ]
  },
}
```

Incremental adoption of Next.js

You can also have Next.js fall back to proxying to an existing website after checking all Next.js routes.

This way you don't have to change the rewrites configuration when migrating more pages to Next.js

JS next.config.js

```
module.exports = {
  async rewrites() {
    return {
      fallback: [
        {
          source: '/*',
          destination: `https://custom-routes-proxying-endpoint.vercel.app/*`,
        },
      ],
    },
  ],
}
```

Rewrites with basePath support

When leveraging `basePath` support with rewrites each `source` and `destination` is automatically prefixed with the `basePath` unless you add `basePath: false` to the rewrite:

JS next.config.js

```
module.exports = {
  basePath: '/docs',

  async rewrites() {
    return [
      {
        source: '/with-basePath', // automatically becomes /docs/with
        destination: '/another', // automatically becomes /docs/another
      },
      {
        // does not add /docs to /without-basePath since basePath: false
        // Note: this cannot be used for internal rewrites e.g. `destination: '/api'`
        source: '/without-basePath',
        destination: 'https://example.com',
        basePath: false,
      },
    ]
  },
}
```

Version History

Version	Changes
v13.3.0	missing added.
v10.2.0	has added.
v9.5.0	Headers added.

[Previous](#)
[< redirects](#)

[Next](#)
[sassOptions](#)